

ROS 2 exam

2h, documents / internet allowed (closed chats please)

1 Description

In this exam you will have to write a C++ node and run it several times through a launch file.

The package should first be compiled, and you can configure the IDE with:

```
cd ~/ros2/src/ecn_2025 # or wherever you have put the repo
colbuild -t # compile this package
ideconf # generate the IDE config file
```

Then the simulation can be run once and for all with:

```
ros2 launch ecn_2025 simulation_launch.py
```

This simulation involves the Baxter robot which has to perform visual servoing with each of its arms. In order to do this, you have to publish a joint velocity command for the arm in order to align the target with the camera frame.

2 Control node

The control node (`control_node.cpp`) should be run for each arm and should:

- Retrieve the pose of the target with regards to the camera
- Retrieve the eJ_e Jacobian through a service
- Compute the control law and publish it as joint velocity

All examples below use the left arm, just replace with `right` for the other limb.

2.1 Relative pose between target and camera

The frames are named as follows:

- the camera frame name is `left_hand_camera`
- the target frame name is `left_target`

Use the provided TF `buffer` in order to check if the transform is available and to get it if it is the case. Note that we are only interested in the `Transform`, while `buffer.lookupTransform` returns a `TransformStamped`. Make sure that the transform you get is in the correct direction (e.g. Z should be positive as the object is in front of the camera).

2.2 Calling the Jacobian service

The simulation proposes a service for each arm to retrieve the current Jacobian. It is of type `baxter_simple_sim::srv::Jacobian`. Use the following command to have an overview of this service:

```
ros2 interface show baxter_simple_sim/srv/Jacobian
```

In our case we want to set the `ee_frame` field to `True` and do not provide any joint position, as we use the current ones.

The service can be called at the following address: `/robot/limb/left/jacobian`.

As in the lab 3, we use a wrapper (namely `ServiceNodeSync`) to have synchronous service calls. This wrapper should:

- be initialized with a unique name and the address of the service through the `init` function
- call the service with the `call` function that returns a `std::optional<Response>`.

If the returned `std::optional` has a value (`response.has_value()`) then the Jacobian is accessed with `response->jacobian`.

2.3 Visual servoing control law

The function `computeVisualServo` takes the pose of the target in the camera frame and the end-effector Jacobian. It returns a `std::vector<double>` for the joint velocity that tracks the object. This value should be written in the command message and published on the well-known topic `/robot/limb/left/joint_command`.

3 The launch file

Once your node runs for a given arm, add a parameter to pick the side (left or right) and run it twice through the `both_launch.py` launch file.

Give two separate names to the node or there might be a name conflict. You do not need to use any namespace, the side parameter should be enough to handle all topics, services and frame names.